

Large Language Models, Fine Tuning, Code Generation

Levente Zsolt, Juhász
Teacher Training Center
University of Dunaújváros
Dunaújváros, Hungary
juhaszle@uniduna.hu

Abstract— Automated software synthesis, text-to-code (code-generation), can be an important aid to software developers. Developments in this area have existed practically since the advent of computing. The new artificial intelligence methods, deep learning (DL), and one type of them, large language models (LLMs) emerged in the last decade, represent a new opportunity for creating this type of application. We survey the transformer technology that underlies the operation of language models. Present paper briefly reviews the topic of software synthesis and code generation, briefly mentioning the main trends. The document concludes with an empirical study demonstrating the described results, where the T5 language model is fine-tuned to generate SQL queries. A fine-tuning set of a few thousand examples train T5 to produce quality queries. The economic principle of diminishing returns (Gossen I) applies, and it is not worth using too large a fine-tuning set. In fact, it is the attractiveness of fine-tuning.

Keywords— *deep learning, transformers, large language models, fine-tuning, code generation*

I. INTRODUCTION

In both software engineering and human-computer interaction (HCI) research, efforts are being made to improve communication between computers and humans. Programming hardware with software, machine code, structural programming, object-oriented programming, and visual programming have all evolved to enable developers to communicate their intentions to the computer at a level of abstraction as close as possible to human thinking. In the case of HCI, the ultimate goal is also to ensure that the user's instructions are understood by the system and that the system performs the appropriate actions. In both these areas, applications that automatically generate program code from text are a great help and an improvement. In natural language processing (NLP), researchers are mainly concerned with the generation of source code, since the program code itself can be considered as special language material. Of course, in software engineering, other levels of software synthesis are also of interest. They do not only aim at producing code at source language level. Besides text-to-code transformation, code generation from other specifications (UML, graphs, diagrams, other programming languages) is also an important focus of studies. In the last 5 years, the development of Artificial Intelligence (AI), as in many other areas of life, opened up many new opportunities. Complex, multi-layered artificial neural networks with special architectures, called transformers, have enabled significant advances in code generation tasks. However, AI still will not replace the work of developers, but it could greatly contribute to the efficiency of their work.

We will describe the operation of the large language models, in particular of the transformer block, which is their principal component. Without striving for completeness this

paper will discuss the problem of code generation and its historically most important approaches. Some of the solutions using language models are underlined and have received considerable attention. From the programming languages, we highlight SQL, the language for managing relational databases, since in the final part of the paper we present an example of the development of a sql-generating system.

II. TRANSFORMERS AND LARGE LANGUAGE MODELS

A. Machine Learning

In the new millennium, one of the key drivers of IT development is the increasing amount of information that can be captured by ubiquitous computing devices (internet, IoT, mobile applications, etc.). The need to deal with data at unprecedented volumes and speeds has led R&D and businesspeople to create and apply new data processing technologies. Big Data is the umbrella term for these new technologies. The abundance of data has also given a new impetus to artificial intelligence (AI) research and development. Maybe now the present fastest growing area of AI is machine learning, which uses statistical learning methods to build software that can learn from data in order to perform certain unprogrammed behaviours. Most of the learning algorithms still in use today were known to researchers in some form since the 1980s, but to achieve performance that now in many cases approaches or even exceeds human performance, a lot of data is needed to teach them. The development of hardware and software technologies and new algorithms were also necessary so that the optimization training of the machines took a tolerable time on these huge datasets. There are 3 broad categories of machine learning, these are supervised learning, unsupervised learning and reinforcement learning [1]. In supervised learning, which is usually a classification or regression task, the algorithm learns by being given 'labelled data' during the training phase. As a result, the program will be able to make estimates for these labels, even in the case of data it has not previously encountered. Examples of supervised learning are face recognition, linear regression or sentiment analysis. In unsupervised learning, the training data is not labelled. The program discovers and learns the relationships and hidden structures between data items. This type includes clustering methods, anomaly and novelty detection, visualization and dimension reduction, association rule learning. Reinforcement learning (RL) can be modelled by Markov Decision Processes: an agent learns over a state space, under the influence of environmental reinforcements, what is the optimal choice (policy) in each state with respect to an aggregate utility function. The main applications of reinforcement learning models are in the control of self-driving systems and in some complex strategy game agent such as AlphaGo [2] player. The performance of systems implemented using these RL models sometimes surpass even the most sophisticated human (world champions') results [3].

B. Deep Learning

In machine learning, the most impressive learning performances are provided by artificial neural network models [4] [5]. These can be part of the solution to any of the machine learning tasks mentioned. As the name of the model suggests, they are composed of connections of abstract processing units (so-called artificial neurons). Learning also means optimization, where the 'weights' of the connections varies. In so-called feed-forward networks, information propagates in one direction, the simplest type being the perceptron, which contains a hidden (intermediate) layer of neurons between the input and output layers (Fig. 1). The brand name for multi-layer neural networks is "Deep Learning" models. Deeper networks generally have better learning performance. The software implementation of artificial neural networks is computed using matrix operations. Both the input to the systems and the connections between the neurons that make up the system are represented in matrix form. The memory of the system is in fact represented by the values (the weights) of the matrices containing the connections between the neurons. During the training, the system tries to change the connection weights based on the size of the difference from the expected results by means of the back-propagation algorithm so that the deviation (the loss) is minimized in the future. The iterative numerical mathematical procedure underlying the algorithm is gradient descent.

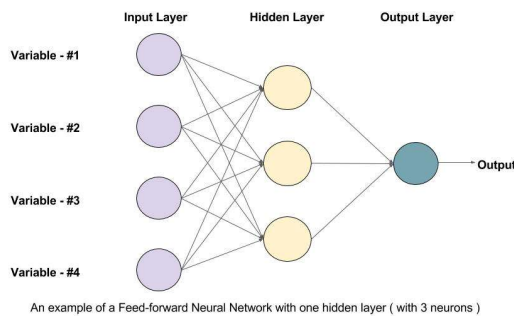


Fig. 1. Feed-forward network [6]

The feature extraction filters of animal "early vision" are modelled and learned by so-called convolutional neural networks (CNNs). They are used in systems implementing machine vision [3] [5].

C. Sequential Networks

Recurrent neural networks are suitable for processing sequential data types and time-series. The networks receive the input in a scheduled manner. For example, in the case of a sentence, per word. For each new item, representations formed from previous states are also fed back into the system (recursion). The network thus has internal state (memory) and is able to process dependencies between the input tokens that may be temporally distant from each other [4] [5]. Sequential networks are thus able to construct, for example, a unique representation of a whole sentence (Fig. 2).

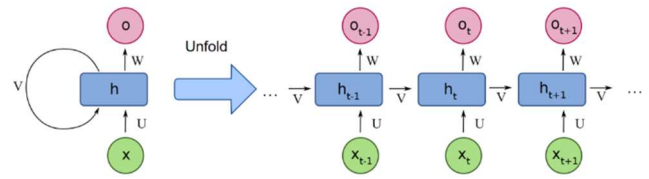


Fig. 2. Recurrent network [7]

The most used types of recurrent networks are GRU (Gated Recurrent Unit) and LSTM (Long Short-Term Memory), in which the models themselves are built from a network of neurons with multiple gating functions. The vector representations of the language units, tokens, formed in such systems are called embeddings. These tokens store the syntactic and semantic properties of words or sentences [6].

D. Language Models

The field of artificial intelligence that deals with the processing and production of human language is natural language processing (NLP), or language technology [6] [7]. There are several related subtasks within NLP: text search, text extraction, information retrieval, sentiment analysis, syntactic-semantic analysis, named entity identification, video/image captioning, speech recognition, and speech generation. The development of dialogue systems (Conversational Artificial Intelligence) is a synthesis task within NLP, since it uses models of both machine language understanding and language generation. In solving NLP problems, deep neural network-based systems generally perform extremely well, often approaching or exceeding human results.

Text-to-speech generative models are usually called language models. They can be considered as functions that can assign a probabilistic value to some sequence of words or tokens. That is, if such a model is trained on material in a given language, it will be able to calculate how frequent a given sequence of words is in that language. This linguistic probabilistic knowledge can be used in a wide range of NLP tasks, including text generation. Purely statistical language models were the earliest attempts to create such models. In these, probability distribution models (so called n -gram models) have been developed mainly based on the frequencies of occurrences of word sequences of different lengths in the learning sample. Since a given word sequence did not necessarily appear in the training set, correction procedures had to be applied to estimate the probability of missing combinations. With these frequency-based methods, the quality of the generated texts was limited [6].

The frequency-based models were replaced by neural network models. In the new millennium, with the continuous development of Deep Learning technology, these models have become more and more prevalent. Initially, models based mainly on the recurrent network architecture were used, which performed better than purely statistical models, but still produced far inferior results to human subjects in various tests measuring the performance of the models. The breakthrough in language model technology came in 2017 with the arriving of transformer architecture [10].

E. Attention

We have seen that the sequential neural networks (or recurrent neural networks, RNN) were the mainstream approach in NLP before transformers. A special RNN

architecture is the decoder-encoder (seq2seq) model. It is composed of two separate trained networks. The first is the encoder part, which builds a machine representation of the tokens (embeddings) based on the language input, creating an internal model of the language. The other part, the decoder, uses these embedding vectors to map the input sequences into output sequences. A typical application for such architectures is machine translation, i.e. sentence-by-sentence or text-by-text automatic language translation (Fig. 3).

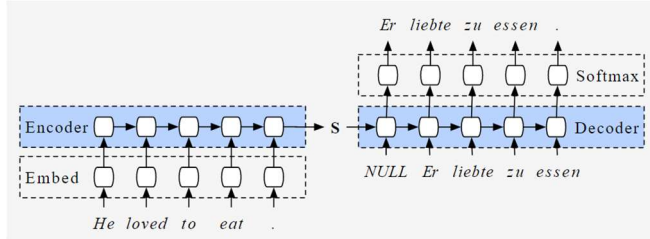


Fig. 3. Seq2seq network. [9]

In the encoder-decoder model, the encoder network created a common vector representation (hidden state) of the whole input (sentence), which was then passed to the decoder network to generate a new text (e.g. the foreign language equivalent of the text). In the decoding part, the generation of the new text is done in an autoregressive way, i.e. in each phase, the network receives, in addition to the embedding vector representing the hidden state, the self-generated token as an input. The generation of the new element is done using the information from these two inputs, up to the generation of the end-of-sentence token. The common state vector, however, behaves as the bottleneck in these systems. Since its length is finite, its information storage capacity is also limited. During processing a sentence, this means that the information about the tokens at the beginning of the sentence remains less accessible at the generative phase of decoding. Thus, the longer dependencies between the words in the input and output sentences could be less exploited by the system. These limitations of the seq2seq models are addressed by the so-called attention mechanism (Fig. 4). On the one hand, the decoding part receives not only the final hidden state vector, but also the state vectors associated with each step of the encoding (first part). Thus, much more contextual information is available for decoding. The attention mechanism allows the decoder to control the weighting (i.e. the amount of "attention") given to each state vector when generating each element. The system also learns these values during training. The attention mechanism can capture the interdependencies between each element of the sentences to be translated and each element of the translated sentences and use them in text generation, whereby the quality of translations is significantly improved [9].

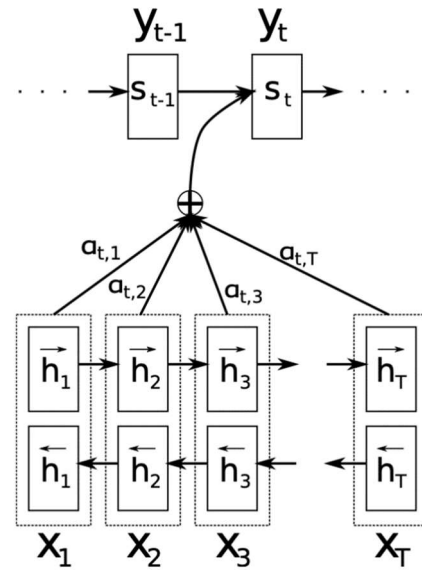


Fig. 4. Attentional mechanisms [9]

F. Transformers, Operations of Transformers, Self-Attention

The transformer model also uses the attention mechanism. However, it does not aim to capture the relationships between input texts and target texts, but rather to acquire the semantic and syntactic relationships between parts of the same input texts (self-attention). It aims to create state representations for each token that depend on the context in which the element is located. This means that the embedding vectors obtained for the same word will differ depending on their occurrence in the text [12].

The breakthrough Transformer model [10], published in 2017, was also based on a seq2seq architecture, i.e. it was built from an encoder and a decoder module (Fig. 5). Both modules consisted of several processing layers. The biggest technological innovation among these was the so-called "multi-head attention layers". The term 'multi-head' refers to the fact that the processing was performed in parallel, essentially independently of each other, the results of which were combined by the system in a later phase. The original Transformer model worked with 8 heads. In the newer transformer-based models, the number of parallel threads is several times more. The parallel operations allow the system to focus on different aspects of the input text and also make it very easy to run the model on parallel hardware (especially on GPUs originally dedicated for graphics processing), thus shortening the training period.

In the attention sublayer, the system works with multiple representations (vectors) of the tokens to be processed. The query vector is the representation of the token being processed. The key vectors are representations of the context elements against which the query vector is compared. The value vectors represent the information content of the tokens. The query, key, and value vectors are generated using weight matrices W_Q , W_K , and W_V , whose values change as the system is trained. The query, key, and value vectors associated with the elements of the input text together form the Q, K, and V matrices. The similarity of vectors can be expressed by their scalar product. On this basis, the relations between tokens are computed using the so-called Scaled Dot-product Attention:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

In this (1) formula, d_k is the scaling factor, the dimension number of the embedded vectors. Scaling (the division) is necessary to maintain the stability of numerical calculations. The softmax function converts the values obtained by multiplication into probability values. The Attention matrix, contains attentional weights, i.e. for each query token, what weight should be given to the elements of the context (value vectors). And multiplying by the vectors of V , we get the contextual representations of the query tokens. These contextual embedded vectors, or their transforms into the embedded vector space, form the output of the attention module. The attention module is also connected to a 'traditional' feed-forward (multi-layer perceptron) network, which performs further processing of the observation representations. The encoding module also includes normalization and residual network functions. The input elements are also supplemented with information about their position in the sequence. The output of an encoder module may be an input to a subsequent similar encoder module or, in the case of a seq2seq architecture, an input to a decoder module performing a generative task.

The building blocks of the decoder module are the same components as those described for the encoder module. The essential difference is that in the case of multi-head attention layers, the elements after the current input token must be masked, the system can only use the elements already received. The first attention layer processes the current input of the decoder ("masked multihead attention") and it is passed to the second attention layer for further processing. It is supplemented by the encoder's word representations, so that the decoder is able to learn the interdependencies between the decoder input texts and the target text (e.g. in the case of translation). This is called cross-attention (Fig. 5).

In order to process the system input as thoroughly as possible, it contains several transformer (encoder-decoder) blocks vertically. This means that the output of the transformer and its constituent encoder and decoder modules will be the input of another transformer module, which will continue to process the representations received from the previous module. The original transformer model had 6 layers, meaning that there were 12 transformer modules in total in the encoder and decoder part. More recent models have multiplied this number [13]. The transformer layers in fact progressively enrich the representation of the words. The early layers capture the simpler, more elementary relations: short-term dependencies, adjacency relations, word order, local syntactic features (word type), simple sentence structure. Higher transformer layers learn semantic information, relations between phrases, thematic roles. And the highest layers of processing grasp abstract relations, discourse structure, holistic emotional meaning (sentiment), and long-distance interdependence relations.

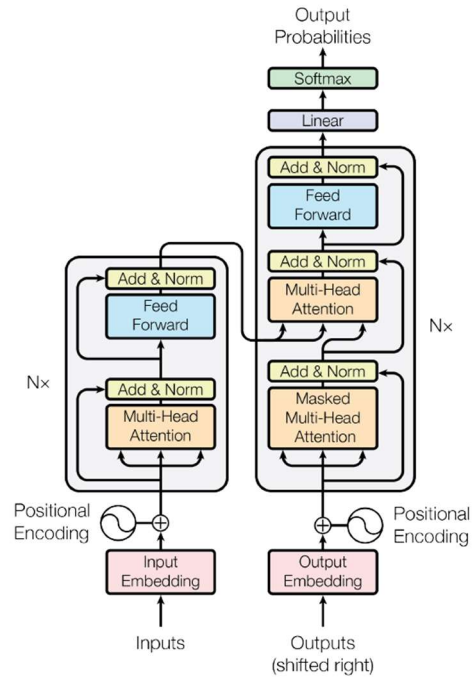


Fig. 5. Transformer [10]

G. Large Language Models (LLMs)

We saw the standard Transformer model consisted of two main parts: decoder and encoder. However, depending on the task for which our system is intended, sometimes one of the two components may be sufficient. There are 3 groups of large language models in terms of architectures: encoder-only, decoder-only, and encoder-decoder models. We illustrate each of these types through some typical representatives of each [14].

Encoder-only models. They work with bidirectional attention: they use elements after the given position in the sentence to construct the word representation. They are auto-encoding systems: they do not need a pre-labelled training set to be trained. Their inputs are masked: certain words are omitted from sentences, and the system has to learn to predict them based on context. The key is to understand the sentences and to develop the right language model. Their primary applications are in sentence classification (e.g. sentiment analysis), entity recognition, and information retrieval. Several language models have been built according to the principles of encoder-only architecture. The best known is BERT (Bidirectional Encoder Representation from Transformer) and its different variations [15]. The BERT has been taught using the aforementioned Masked Language Model and the Next Sentence Prediction Task method. In the latter, the task of the system was to decide, given two sentences, whether the first sentence follows the second. The large BERT model contained 12 heads and 24 encoder layers. It had 340 million trainable parameters (today it is no longer considered a "large" model).

Decoder-only models. They are autoregressive models: during training, their input is the output token generated by themselves in the previous phase. The attention layer only "sees" the positions preceding the current input: learning is only done in one direction. Learning tasks are prediction made on the next element in the input sequence. Most common

applications: language content generation and question-answering systems. Members of the GPT (Generative Pre-trained Transformer) family are the best-known representatives of encoder-only type transformers. The GPT-3 contains 96 decoder modules and 96 heads. The number of trainable parameters is 175 billion [16].

Encoder-decoder models. This architecture used for seq2seq processing similarly to the original Transformer model. The attention layer of the decoder accesses the whole input sequence, the encoder does only the parts preceding this position. They are auto-encoding and auto-regressive systems with the same learning algorithms we have seen for separated systems. Their application is mainly in tasks where input sequences (texts) need to be converted into output texts. E.g. machine translation, content summarization, generative response generation. Typical representatives of Encoder-decoders are T5 [17] and BART [17]. The BART (Bidirectional and Auto-Regressive Transformers) model is a combination of the BERT and GPT-2 models, which learns the context of the text in both directions and uses masked language modelling in both the encoder and decoder parts. It performs well in abstract summarization, natural language generation, and language inference tasks, among others. 400 million parameters, 16 heads, 12 transformer modules in the encoder and 12 transformer modules in the decoder. The T5 (Text-To-Text Transfer Transformer) model uses a unified framework that treats all language tasks as text-to-text transformations. Both of its parts use a masked language model. The T5 model has achieved outstanding results in more than 20 language tasks such as machine translation, summarization, question-answering, and common-sense understanding. The t5-tb sub-version contains 24-24 transformer modules with 128 heads and 11 billion parameters.

H. Foundational models and fine-tuning

Many of the trained transformer models are also available for external users. The largest collection of these can be found on Hugging Face [19]. In some cases, these have been trained using only exercises similar to the text completion described above. These are called foundational models, and they require additional fine-tuning to solve more specific language technology tasks [20]. In the machine learning literature, what we mean by transfer learning is when we use a previously trained model to train an existing system to perform something new [21]. Fine-tuning usually involves adding additional layers to the existing system depending on the new task. During fine-tuning, or retraining, the system is given input data corresponding to the new task, and training is performed on this data. During the process, training may also involve the existing layers of the system, but this is usually almost as resource intensive as teaching a new model. More often, some or even all of the layers of the base model are "frozen", i.e. they cannot be changed during post-training. In this case, only the non-fixed and additional layers are learned. The effectiveness of re-training is usually proportional to the number of layers involved, but for efficiency reasons, the whole network is rarely used. The methods that can be used to significantly reduce the costs of transfer learning are called parameter-efficient fine-tuning (PEFT) [22]. In the case of additive methods, virtual elements are concatenated to the model, and only these can be trained during fine-tuning. In the case of prompt tuning, a new virtual token is added to the input layer, while in the case of prefix tuning, it is added to all attention blocks. In the reparameterization procedures, the

parameter matrix is factored into matrices of lower rank. During fine-tuning, these lower rank matrices must be updated. It can mean even 96% reduction in number of trainable parameters.

In the case of ChatGPT [23], the base foundational model was the OpenAI GPT-3.5 decoder model. It was fine-tuned to be able to answer questions, generate stories, add program code, among other things. The method used was Reinforcement Learning from Human Feedback (RLHF): the system learned from the responses of users and from the human evaluation of the system's responses.

I. Prompting, prompt engineering

You can improve the performance of your models by more than just fine-tuning. During in-context learning, prompt-response examples are presented to the system. We also expect such answers from the system [24]. In this case, the system's state (its parameters) is not updated, it only receives additional context for the task, based on which it can generate higher quality responses. In zero-shot learning, the system produces high-quality answers without an example prompt. N-shot or few-shot learning requires one or more examples to be presented by the user. Prompt engineering is the term used to describe procedures to increase the quality of the responses received from the system. In the case of ChatGPT, for example, it helps to specify the purpose of the query, the context and other additional information, the role (perspective) the system should take (e.g. as a teacher, as a layperson, etc.) and the constraints (length of the response, style, language, etc.). For computational and logical reasoning tasks, the system is more successful if it is asked to perform the operation in steps and to justify its choices (Chain of Thought reasoning).

III. CODE-GENERATION

AI already supports the software development process in many areas. The tasks of AI-assisted programming are as follows: code generation, code completion, code translation, code refinement, code summarization, defect detection, clone detection [25]. The most popular assistant applications are GitHub Copilot, ChatGPT, Gemini, CodeGPT.

The tools used for automatic code generation generally reflected the state of the art of Artificial Intelligence at the time. In their summary, Ghaemi et al [26] distinguished 3 groups of models: None-learning, Shallow learning, Deep learning.

The none-learning models included rule-based systems. Early attempts were heuristics using manually written rules to generate program code from some specification or textual description. Code generation from UML diagrams [27], Domain Specific Languages [28] and Template-Based Code Generation (TBCG) [29], still popular in web and mobile development, follow in this tradition. Some none-learning systems can be classified as probabilistic grammar-based models [30] [31]. The shallow learning group includes n-gram and Hidden Markov based language models [32]. The Deep learning group can be further classified based on network-type [26]: RNN-based models, Non-RNN based models, Attention based models, and Transformer based models. RNN-based group includes models with LTSM (Code2seq, [33]) and GRU architecture (CodeGRU, [34]).

The first Transformer-based code generation LLM model (with BERT architecture) was CodeBERT [35].

Austin [36] investigated the program synthesis capability of LLMs of different sizes (244m - 137B parameters) on two text-to-code datasets. The quality of the generated elementary-level programs is loglinearly dependent on the size of the model. The largest base model solved elementary Python problems with an efficiency of nearly 60% and simple mathematical programming problems with an efficiency of 84%. Fine-tuning increased performance by 10%, and human feedback also increased performance. However, for more complex tasks or those with new inputs, performance deteriorated.

Codex [37] GPT-based fine-tuned model. The HumanEval task set was used. Complex tasks requiring multiple steps of inference were difficult to handle.

The AlphaCode [38] encoded-decoder structured language model was trained on code and commentaries found on GitHub. The AlphaCode model was fine-tuned on textual descriptions and solutions of programming contest tasks from the CodeForces [39] platform. For each problem, the program generated a solution set and then selected the top 10 candidate solutions using a filtering (syntax errors, program execution gives the expected result) and clustering (chooses the simplest from similar solutions) procedure. The solutions generated by AlphaCode would rank 54.3% in a real competition. The base model of AlphaCode 2 [39] is already Gemini Pro [40], and this has also been fine-tuned on CodeContest [39] exercises. The new model has already achieved a ranking of 85%.

The number of pretrained code generation models is constantly growing. In their summary, [26] reported a total of 18 such models. Nowadays, fine-tuning is routinely performed, and in many cases one of the methods of Parameter efficient fine-tuning (PEFT) is used: adapters, LoRA, prompt tuning, prefix tuning. The transformers used for code generation are usually of the decoder-only or encoder-decoder type. There are two exceptions, one being the already mentioned CodeBERT [35] and its improved version Graphcodebert [42]. In the decoder-only group, the best-known models are of course those based on members of the GDP family. For example, CodeGPT [43] or the already mentioned Codex [37] [43]. There are also fine-tuned versions of the Meta Llama [44] language model that can also be used for code generation, such as CodeLlama [45] or Lemur [46]. Among the encoder-decoders, the T5 and BART models already described in the previous section are the most well-known.

There are several datasets containing computer code and general text that are used to pretrain foundation models. Among these, the largest are Stack (3TB), Pile (1.25TB), ROOTS (1.6TB) [47] [48] [49]. There are two groups of metrics for evaluating text-to-code models. One, match-based evaluation, is the comparison of generated and expected codes. Examples include BLEU and ROUGE, which are popularly used in generative NLP tasks. Execution-based evaluation compares the result of running the code with the expected result [26].

Text-to-sql tasks can be considered a subtype of code generation. Since in the last section we will teach an LLM to

generate SQL code with fine-tuning, it is worth mentioning this separately. SQL (Structured Query Language) is a domain-specific language, known primarily as the programming language of relational database management systems (RDBMS), which is very useful for dealing with structured data, data that contains relations between variables and entities. Its mathematical basis is relational algebra and tuple relational calculus. The language contains query, definition, control, and manipulation submodules. It is essentially declarative, but also contains procedural elements. It was standardised in 1978 (ISO). Some question the status of SQL as a programming language. Nevertheless, it is currently ranked 10th in the TIOBE index. It is Turing-complete, which is implemented using recursive common table expressions (Reprintsev, 2018 Silberschatz 2020). SQL has several dialects.

Similar to general code generation tasks, a number of rule-based methods have been developed previously (e.g., [50] [51]). Then, with the advent of the Deep Learning era, text-to-sql models of this type appeared (e.g. [38] [52]). Today's solutions are almost exclusively implemented using some form of LLM with fine-tuning or in-context learning. As summarized by [53], the LLM used in in-context learning tasks is in most cases GPT-4 or ChatGPT, with a more modest representation by Codex. The best known text-to-sql dataset is Spider [54], but other data are also available via the HuggingFace data hub. Text-to-sql solutions can be evaluated on the basis of content matching, i.e. the extent to which the elements or components of the generated SQL are identical to the expected gold answer. And execution-based metrics consider the match between the search produced by the generated answer and the expected results of the text [54] [55].

IV. FINE-TUNING EXPERIMENT

We fine-tuned a language model, T5. After the fine-tuning process, the system is able to solve a text-to-sql task. In the task, a prompt is given, which contains the requested query expressed in natural language and, as the context of the query, the definition of the underlying database table in SQL. The system will generate the corresponding SQL query. The most important advantage of the fine-tuning procedure is that it requires a training set with a much smaller number of elements and a much shorter training time to achieve results. In this study we tested the effect of different sizes of training sets on the learning of the text-to-sql task.

Foundational Model. The T5 label is a double acronym (Text-to-Text Transfer Transformer). This name was given to an early language model developed by Google AI [17]. We have already seen that its structure is seq2seq (or encoder-decoder), similar to the classic Transformer model (Fig. 5). It transforms text-based input into output text. The pre-training was based on the C4 [56] dataset, which consists mainly of scraped text and code material from the internet. It contains about 750 GB of uncompressed text, including about 350 million English-language web pages. The latest version of the model, T5X, has been trained on a larger (826 GB) and more heterogeneous 'pile' dataset, containing material not only from the web. T5 has also been fine-tuned for a range of downstream tasks, including machine translation, text compression, chatbots, and code generation (Fig. 6).

TABLE I. T5 MODEL VARIANTS IN DIFFERENT SIZES (SOURCE: [57])

Model	Parameters	# layers	d_{model}	d_{ff}	d_{kv}	# heads
Small	60M	6	512	2048	64	8
Base	220M	12	768	3072	64	12
Large	770M	24	1024	4096	64	16
3B (XL)	3B	24	1024	16384	128	32
11B (XXL)	11B	24	1024	65536	128	128

The T5 model is available in several sizes (Table 1). In the table:

- # layers: Number of layers in the encoder; also, number of layers in the decoder. They always have the same number of layers.
- # heads: Number of attention heads in each attention block.
- d_{model} : Dimension of the embedding vectors.
- d_{ff} : Dimension of the feedforward network within each encoder and decoder layer.
- d_{kv} : Dimension of the key and value vectors used in the self-attention mechanism.

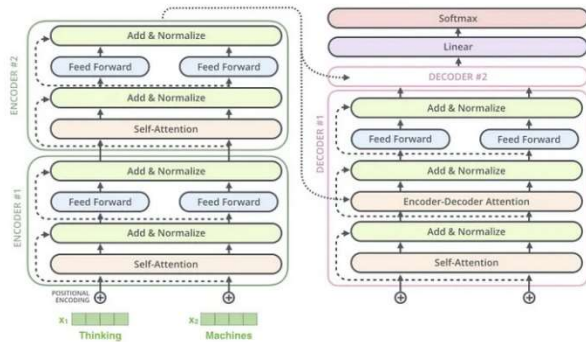


Fig. 6. The structure of T5 (source: [17])

Due to limited hardware resources, the small model was chosen.

Data. 3 publicly available (via Huggingface data-hub) text-to-sql datasets were used [58] [59] [60]. Each of them contained the columns required for training: a verbally formulated query, a table definition SQL command given as context, and a golden data, i.e. the expected reference answer. The structure of the data frames differed slightly, so they had to be standardized during pre-processing and then the three data sets had to be standardized. The whole dataset consisted of 148300 examples. Of these, 80% went into the training set and 10%-10% into the validation and training sets, respectively.

Metric. ROUGE (Recall-Oriented Understudy for Gisting Evaluation) was applied [61]. It was originally developed for evaluation of automatic summarization, machine translation tasks. It compares a generated text to one or more reference summaries or translations. Actually, it compares the n-grams of the generated text to the n-grams of the references.

- Rouge-1 recall = number of words matches / number of words in reference.
- Rouge-1 precision = number of words matches / number of words in generated text.
- Rouge-1 F1-score = $2(\text{precision} * \text{recall} / (\text{precision} + \text{recall}))$.
- Rouge-2 recall = number of overlapping bigrams / number of bigrams in reference.
- Rouge-L: longest common subsequence / number of words in reference or generated text.
- Rouge-Lsum: average of all Rouge-Ls (per sentence).

The rouge implementation of the transformers python package (Huggingface) returns the recall values.

Process. The first step in preprocessing the data was to create a single unique set. The recommendations of [62] were used in choosing appropriate preprocessing methods and training hyperparameters throughout the experiment. Finally, the training, validation and test sets were separated. For the fine-tuning, the elements of the training set were used to create the input prompt with the natural language query, SQL statement context and the expected reference answer. This was converted into number vectors using the tokenization class for the T5 model. The fine-tuning was performed with different training set sizes: 1000, 5000, 15000, 30000, 60000, and 120000. The efficiency of the fine-tuning was estimated using the Rouge measure.

Hardware. Fine-tuning experiments were performed on the Google Colab Pro+ platform. It also allows access to Pro+ high-performance GPUs, which can be subscribed to, in addition to the otherwise free service. In our case, it is a high-performance A100-SXM4-40GB GPU, with 83GB RAM and 40GB VRAM memory. Running times can be found in the results section.

Software. Python programming language, pytorch and transformers and other data science packages. Colab Pro+, Jupyter Notebook.

Results (Table 1). Even at a size of 5000, excellent rouge values (above 0.5) were obtained (Fig 7), so it is not worth expanding the learning set further. The economic principle of diminishing returns (Gossen I) applies [63], and it is not worth using too large a fine-tuning set. In fact, it is the attractiveness of fine-tuning. The T5 pre-training sets (either C4 or Pile) contained a lot of code, including SQL code. It is certain that the system did not acquire the SQL knowledge for the surprisingly high rouge values during fine-tuning. It caught during the pretraining. However, in order to solve text-to-sql tasks, fine-tuning was necessary.

TABLE II. THE FINE-TUNING RESULTS. RUNNING TIMES, ROUGE METRICS, TRAINING AND VALIDATION LOSSES

N	walltime	rouge1	rouge2	rougeL	rougeLsum	Training Loss	Validation Loss
0	0	0.076	0.015	0.066	0.066		
1000	23.6s	0.161	0.051	0.157	0.157		
5000	122s	0.88	0.81	0.86	0.86	0.039	0.035
15000	351s	0.91	0.86	0.9	0.9	0.024	0.023
30000	721s	0.93	0.89	.92	.92	0.023	0.017
60000	1497s	0.94	0.91	0.93	.93	0.017	0.014
120000	3056s	0.92	0.89	0.91	.91	0.014	0.013

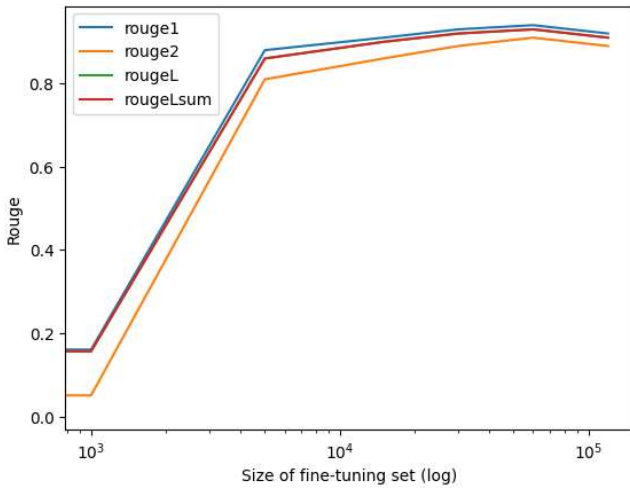


Fig. 7. Dependence of Rouge values on the size of the training sets (log N)

The operation of fine-tuning is obviously influenced by other factors besides the size of the fine-tuning set. Further interesting questions can be investigated regarding the effect of the previously briefly mentioned Parameter-Efficient Fine-Tuning (PEFT) methods. The quality of the generated sql commands should also be examined in further investigations. This would primarily require the involvement of external expert evaluators.

REFERENCES

- [1] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems. 3rd edition, O'Reilly Media, 2022.
- [2] D. Silver, A. Huang, C. J. Maddison et al., „Mastering the game of Go with deep neural networks and tree search,” *Nature*, pp. 484-489, 2016.
- [3] S. Russell és P. Norvig, Artificial Intelligence: A Modern Approach 4th Edition, Pearson, 2021.
- [4] I. Goodfellow, Y. Bengio és A. Courville, Deep Learning (Adaptive Computation and Machine Learning series), The MIT Press, 2016.
- [5] A. Zhang, Z. C. Lipton, M. Li és A. J. Smola, Dive into Deep Learning, Cambridge University Press, 2023.
- [6] „learnopencv.com” 17 szeptember 2023. [Online]. Available: <https://learnopencv.com/understanding-feedforward-neural-networks/>.
- [7] „www.analyticsvidhya.com,” 17 szeptember 2023. [Online]. Available: <https://www.analyticsvidhya.com/blog/2022/03/a-brief-overview-of-recurrent-neural-networks-rnn/>.
- [8] D. Jurafsky és J. H. Martin, Speech and Language Processing (3rd ed. draft), 2023.
- [9] J. Eisenstein, Dive into Deep Learning, The MIT Press, 2019.
- [10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser és I. Polosukhin, Attention Is All You Need, arXiv:1706.03762, 2017.
- [11] D. Bahdanau, K. Cho és Y. Bengio, Neural Machine Translation by Jointly Learning to Align and Translate, arXiv:1409.0473, 2014.
- [12] S. Yildirim és M. Asgari-Chenaghlu, Mastering Transformers: Build state-of-the-art models from scratch with advanced natural language processing techniques, Packt Publishing, 2021.
- [13] L. Tunstall, L. v. Werra és T. Wolf, Natural Language Processing with Transformers, Revised Edition 1st Edition, O'Reilly Media, 2022.
- [14] D. Rothman és A. Gulli, Transformers for Natural Language Processing: Build, train, and fine-tune deep neural network architectures for NLP with Python, Hugging Face, and OpenAI's GPT-3, ChatGPT, and GPT-4 2nd ed. Edition, Packt Publishing, 2022.
- [15] J. Devlin, M.-w. Chang, K. Lee és K. Toutanova, BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, %1. Vol. arXiv:1810.04805v2, 2018.
- [16] T. e. a. Brown, Language models are few-shot learners. Advances in neural information processingsystems, arXiv:2005.14165, 2020.
- [17] C. Raffel, N. Shazeer, A. Roberts, K. Lee, M. Narang, M. Matena, Y. Zhong, W. Li és P. J. Liu, „Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” arXiv preprint , p. arXiv:1910.10683, 2019.
- [18] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov és L. Zettlemoyer, BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension, arXiv:1910.13461, 2019.
- [19] „Hugging Face,” 17 szeptember 2023. [Online]. Available: <https://huggingface.co/>.
- [20] X. W. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong és J.-R. Wen, „A Survey of Large Language Models,” arXiv:2303.18223 [cs.CL], 2023.
- [21] P. Azunre, Transfer Learning for Natural Language Processing, Manning, 2021.
- [22] L. Xu, H. Xie, S.-Z. J. Qin, X. Tao és F. L. Wang, „Parameter-Efficient Fine-Tuning Methods for Pretrained Language Models: A Critical Review and Assessment,” arXiv:2312.12148, 2023.
- [23] OpenAI, „GPT-4 Technical Report,” arXiv:2303.08774 [cs.CL], 2023.
- [24] Q. Dong, L. Li, D. Dai, C. Zheng, J. Ma, R. Li, H. Xia, J. Xu, Z. Wu, B. Chang, X. Sun, L. Li és Z. Sui, „A Survey on In-context Learning,” arXiv:2301.00234 .
- [25] M.-F. Wong, S. Guo, C.-N. Hang, S.-W. Ho és C.-W. Tan, „Natural Language Generation and Understanding of Big Code for AI-Assisted Programming: A Review,” *Entropy*, 25, p. 888., 2023.
- [26] H. Ghaemi, Z. Alizadehsani, A. Shahraki és J. M. Corchado, „Transformers in source code generation: A comprehensive survey,” *Journal of Systems Architecture*, Vol. 153, 2024.
- [27] E. Sunitha és P. Samuel, „Automatic code generation from UML state chart diagrams,” *IEEE Access*, 7, p. 8591-8608., 2019.
- [28] R. Alur, G. Bodik, G. Juniwal és et al., „Syntax-Guided Synthesis,” IEEE, 2013.

- [29] B. Uyanik és V. H. Sahin, „A template-based code generator for web applications,” *Turkish Journal of Electrical Engineering and Computer Sciences*, Vol. 28, 1. 3, 2020.
- [30] P. Bielik, V. Raychev és M. Vechev, „PHOG: probabilistic model for code,” in *International Conference on Machine Learning*, PMLR, 2016, pp. 2933-2944.
- [31] T. Cohn, P. Blunsom és S. Goldwater, „Inducing tree-substitution grammars,” *Journal of Machine Learning Research*, Vol. 11, pp. 3053-3096, 2010.
- [32] V. Raychev, M. Vechev és E. Yahav, „Code completion with statistical language models,” in *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2014, p. 419–428.
- [33] U. Alon, S. Brody, O. Levy és E. Yahav, „Code2seq: Generating sequences from structured representation of code,” in *International Conference on Learning Representation*, 2018.
- [34] Y. Hussain, Z. Huang, Y. Zhou és S. Wang, „CodeGRU: Context-aware deep learning with gated recurrent unit for source code modeling,” *Information and Software Technology*, Vol. 125, p. 106309, 2020.
- [35] Z. Feng, D. Guo, D. Tang és et al., „CodeBERT: A Pre-Trained Model for Programming and Natural Languages,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, Association for Computational Linguistics., 2020, pp. 1536-1547.
- [36] J. Austin, A. Odena, M. Nye és et al., „Program Synthesis with Large Language Models,” arXiv:2108.07732, 2021.
- [37] M. Chen, J. Tworek, H. Jun és et al., „Evaluating Large Language Models Trained on Code,” arXiv:2107.03374.
- [38] Y. Li, D. Choi, J. Chung és et al., „Competition-level code generation with AlphaCode,” *Science*, Vol. 378 6624, pp. 1092-197, 2022.
- [39] „Codeforces,” [Online]. Available: <https://codeforces.com/>. [19 July 2024].
- [40] G. D. AlphaCode Team, „AlphaCode 2 Technical Report,” 2023.
- [41] G. Gemini Team, „Gemini: A Family of Highly Capable Multimodal Models,” 2023.
- [42] D. Guo, S. Ren, S. Lu, Z. Feng, Tang és et al., „Graphcodebert: Pre-training code representations with data flow,” in *ICLR, International Conference on Learning*, 2021.
- [43] S. Lu, D. Guo, S. Ren és et al., „CodeXGLUE: A machine learning benchmark dataset for code understanding and generation,” *CoRR*, p. abs/2102.04664, 2021.
- [44] H. Touvron, T. Lavril, G. Izacard és et al., „LLaMA: Open and Efficient Foundation Language Models,” arXiv, p. arXiv:2302.13971, 2023.
- [45] B. Roziere, J. Gehring, F. Gloeckle és et al., „Code Llama: Open foundation models for code,” arXiv preprint, p. arXiv:2308.12950., 2023.
- [46] X. Xu, H. Su, C. Xing és et al., „Lemur: Harmonizing natural language and code for language agents,” arXiv, p. arXiv:2310.06830, 2023.
- [47] D. Kocetkov, R. Li, L. B. alla és et al., „The Stack: 3 TB of permissively licensed source code,” *Trans. Mach. Learn. Res.*, 2013.
- [48] L. Gao, S. Biderman, S. Black, L. Goldind, T. Hoppe és et al., „The Pile: An 800GB Dataset of Diverse Text for Language Modeling,” arXiv, p. 2001.00027, 2020.
- [49] H. Laurencon, L. Sulnier, T. Wang és et al., „The bigscience roots corpus: A 1.6 tb composite multilingual dataset,” *Adv. Neural Inf. Process. Syst.*, Vol. 35, p. 31809–31826., 2022.
- [50] F. Li és H. V. Jagadish, „Constructing an interactive natural language interface for relational databases,” *Proceedings of the VLDB Endowment*, %1. Vol. 8(1), pp. 73-84, 2014.
- [51] T. Mahmud, K. A. Hasan, M. Ahmed és T. H. C. Chak, „rule based approach for nlp based query processing,” *International Conference on Electrical Information and Communication Technologies (EICT)*, 2015.
- [52] B. Hui, X. Shi, R. Geng és et al., „Improving text-to-sql with schema dependency learning,” arXiv, p. arXiv:2103.04399, 2021.
- [53] Z. Hong, Z. Yuan, Q. Zhang és et al., „Next-Generation Database Interfaces: A Survey of LLM-based Text-to-SQL,” arXiv, p. arXiv:2406.08426v2, 2024.
- [54] T. Yu, R. Zhang, K. Yang és et al., „Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task,” in *Empirical Methods in Natural Language Processing (EMNLP)*, 2018.
- [55] M. Poureza és D. Raffei, „Evaluating cross-domain text-to-SQL models and benchmarks,” in *Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [56] C. Raffel, A. Shazeer, A. Robert és et al., „Colossal Clean Crawled Corpus (C4),” Hugging Face, 2020.
- [57] Wikipedia, „T5,” [Online]. Available: [https://en.wikipedia.org/wiki/T5_\(language_model\)#](https://en.wikipedia.org/wiki/T5_(language_model)#).
- [58] b-mc2, „sql-create-context Dataset,” 2023.
- [59] Clinton, „Text-to-sql-v1 dataset”.
- [60] R. Tiwari, „Know sql dataset”.
- [61] C.-Y. Lin, „ROUGE: A Package for Automatic Evaluation of Summaries,” *Proceedings of the Workshop on Text Summarization Branches Out (WAS 2004)*, 2004.
- [62] M. Keywood, „Fine tuning a T5 small model to generate SQL from natural language with 92.3% accuracy,” 2024. [Online]. Available: <https://medium.com/@martinkeywood/fine-tuning-a-t5-small-model-to-generate-sql-from-natural-language-with-92-3-accuracy-fb29e062c638>. [19 July 2024].
- [63] wikipedia, „Diminishing returns,” [Online]. Available: https://en.wikipedia.org/wiki/Diminishing_returns. [19 July 2024]
- [64] D. Choi, M. C. Shin, E. Kim és D. R. Shin, „Ryan-sql: Recursively applying sketch-based slot fillings for complex text-to-sql in cross-domain databases,” *International Conference on Computational Linguistics (COLING)*, 2021.

